Code:

Exp np 26

```
#include <stdio.h>

int main() {
   int a[50], b[50], merged[100];
   int n1, n2, i, j;

   // Input size and elements for first array
   printf("Enter size of first array: ");
   scanf("%d", &n1);

   printf("Enter elements of first array:\n");
   for(i = 0; i < n1; i++) {
      scanf("%d", &a[i]);
   }

   // Input size and elements for second array
   printf("Enter size of second array: ");
   scanf("%d", &n2);

   printf("Enter elements of second array:\n");
   for(i = 0; i < n2; i++) {
      scanf("%d", &b[i]);
   }

   // Merge arrays
   for(i = 0; i < n1; i++) {
      merged[i] = a[i];
   }
```

```
    for(j = 0; j < n2; j++) {
        merged[i] = b[j];
        i++;
    }

    // Display merged array
    printf("Merged array: ");
    for(i = 0; i < n1 + n2; i++) {
        printf("%d ", merged[i]);
    }

    return 0;
}
```

Output:

```
Enter size of first array: 4
Enter elements of first array:
10 20 30 40
Enter size of second array: 3
Enter elements of second array:
3
30 40 50
Merged array: 10 20 30 40 3 30 40
```

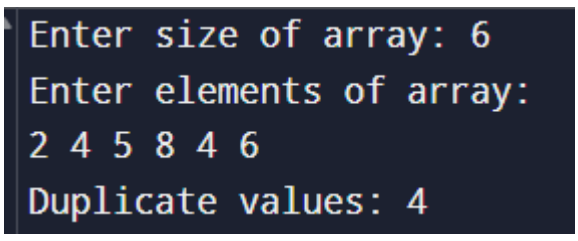
Exp no 27:

Code:

```
#include <stdio.h>

int main() {
    int arr[50], n, i, j;

    printf("Enter size of array: ");
```

```
    scanf("%d", &n);

    printf("Enter elements of array:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Duplicate values: ");
    int found = 0;
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(arr[i] == arr[j]) {
                printf("%d ", arr[i]);
                found = 1;
                break;
            }
        }
    }

    if(!found) {
        printf("No duplicates found");
    }

    return 0;
}
```

OUTPUT:



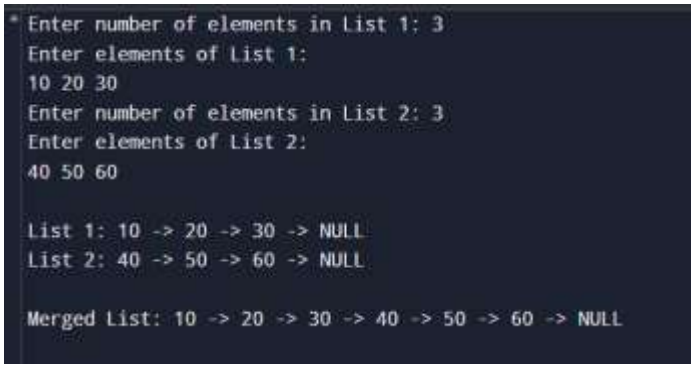
EXP NO 28:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Structure for node
struct Node {
    int data;
    struct Node* next;
};

// Function to create new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at end
void insertEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
```

```
// Function to merge two linked lists
struct Node* mergeLists(struct Node* head1, struct Node* head2) {
    if (head1 == NULL) return head2;
    struct Node* temp = head1;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = head2;   // Attach list2 at the end of list1
    return head1;
}

// Function to display list
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head1 = NULL, *head2 = NULL;
    int n1, n2, data, i;

    printf("Enter number of elements in List 1: ");
    scanf("%d", &n1);

    printf("Enter elements of List 1:\n");
    for(i = 0; i < n1; i++) {
        scanf("%d", &data);
        insertEnd(&head1, data);
```

```
    }

    printf("Enter number of elements in List 2: ");
    scanf("%d", &n2);

    printf("Enter elements of List 2:\n");
    for(i = 0; i < n2; i++) {
        scanf("%d", &data);
        insertEnd(&head2, data);
    }

    printf("\nList 1: ");
    display(head1);

    printf("List 2: ");
    display(head2);

    // Merging
    head1 = mergeLists(head1, head2);

    printf("\nMerged List: ");
    display(head1);

    return 0;
}
```



EXP NO 29:

CODE:

```
#include <stdio.h>
#define INF 9999
#define MAX 10

void dijkstra(int G[MAX][MAX], int n, int start) {
   int cost[MAX][MAX], distance[MAX], visited[MAX], pred[MAX];
   int count, mindistance, nextnode, i, j;

   // Create cost matrix
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         if (G[i][j] == 0)
            cost[i][j] = INF;
         else
            cost[i][j] = G[i][j];

   // Initialize
   for (i = 0; i < n; i++) {
      distance[i] = cost[start][i];
      pred[i] = start;
      visited[i] = 0;
   }

   distance[start] = 0;
   visited[start] = 1;
   count = 1;

   while (count < n - 1) {
      mindistance = INF;
```

```
    // Next minimum distance node
    for (i = 0; i < n; i++)
        if (distance[i] < mindistance && !visited[i]) {
            mindistance = distance[i];
            nextnode = i;
        }

    visited[nextnode] = 1;

    for (i = 0; i < n; i++)
        if (!visited[i])
            if (mindistance + cost[nextnode][i] < distance[i]) {
                distance[i] = mindistance + cost[nextnode][i];
                pred[i] = nextnode;
            }
    count++;
}

// Print shortest distance and path
for (i = 0; i < n; i++) {
    if (i != start) {
        printf("\nDistance to node %d = %d", i, distance[i]);
        printf("\nPath = %d", i);

        j = i;
        while (j != start) {
            j = pred[j];
            printf(" <- %d", j);
        }
    }
```

```
        printf("\n");
    }
}

int main() {
    int G[MAX][MAX], n, i, j, start;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter weighted adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &G[i][j]);

    printf("Enter starting node: ");
    scanf("%d", &start);

    dijkstra(G, n, start);

    return 0;
}
```

```
Enter number of nodes: 3
Enter weighted adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter starting node: 0
```

EXP NO 30:

CODE:

```
#include <stdio.h>
#define INF 9999
#define MAX 10

int n;
int cost[MAX][MAX];
int visited[MAX];
int minCost = INF;

void tsp(int city, int count, int currentCost, int start) {
    if (count == n && cost[city][start] > 0) {
        int totalCost = currentCost + cost[city][start];
        if (totalCost < minCost) {
            minCost = totalCost;
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[city][i] > 0) {
            visited[i] = 1;
            tsp(i, count + 1, currentCost + cost[city][i], start);
            visited[i] = 0;
        }
    }
}

int main() {
    printf("Enter number of cities: ");
```

```
    scanf("%d", &n);

    printf("Enter cost adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    tsp(0, 1, 0, 0);

    printf("\nMinimum cost for Traveling Salesman Path = %d\n", minCost);

    return 0;
}
```

OUTPUT:

```
Enter number of cities: 4
Enter cost adjacency matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

EXP NO 31:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
```

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Inorder Traversal (Left, Root, Right)
void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

// Preorder Traversal (Root, Left, Right)
void preorder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

// Postorder Traversal (Left, Right, Root)
```

```
void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int main() {
    // Creating test binary tree
    struct Node* root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    printf("Inorder Traversal: ");
    inorder(root);

    printf("\nPreorder Traversal: ");
    preorder(root);

    printf("\nPostorder Traversal: ");
    postorder(root);

    return 0;
}
```

OUTPUT:

```
Inorder Traversal: 40 20 50 10 30
Preorder Traversal: 10 20 40 50 30
Postorder Traversal: 40 50 20 30 10
```

EXP NO 32:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure of a Doubly Linked List
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

struct Node* head = NULL;

// Function to insert a node at end
void insertEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
```

```
    newNode->prev = temp;
}

// Display list forward
void displayForward() {
    struct Node* temp = head;
    printf("Forward List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Display list backward
void displayBackward() {
    struct Node* temp = head;
    if (temp == NULL) return;

    // Move to last node
    while (temp->next != NULL)
        temp = temp->next;

    printf("Backward List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}
```

```
int main() {
    int n, value;

    printf("Enter number of elements to insert: ");
    scanf("%d", &n);

    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        insertEnd(value);
    }

    displayForward();
    displayBackward();

    return 0;
}
```

OUTPUT:

```
Enter number of elements to insert: 4
Enter values:
10 20 30 40
Forward List: 10 <-> 20 <-> 30 <-> 40 <-> NULL
Backward List: 40 <-> 30 <-> 20 <-> 10 <-> NULL
```

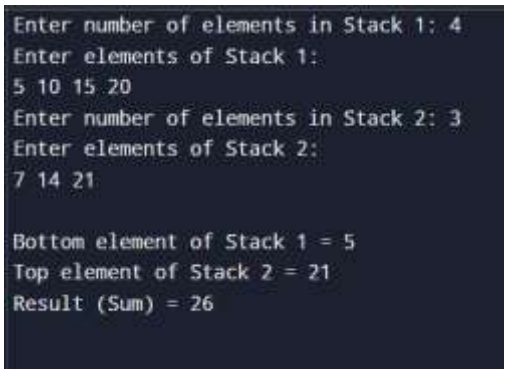
EXP NO 33:

CODE:

```
#include <stdio.h>

int main() {
    int stack1[50], stack2[50];
    int n1, n2, i;
    int bottom1, top2, result;
```

```
    // Input stack 1
    printf("Enter number of elements in Stack 1: ");
    scanf("%d", &n1);
    printf("Enter elements of Stack 1:\n");
    for(i = 0; i < n1; i++)
        scanf("%d", &stack1[i]);

    // Input stack 2
    printf("Enter number of elements in Stack 2: ");
    scanf("%d", &n2);
    printf("Enter elements of Stack 2:\n");
    for(i = 0; i < n2; i++)
        scanf("%d", &stack2[i]);

    // Bottom of stack1 → first element (index 0)
    bottom1 = stack1[0];

    // Top of stack2 → last element (index n2-1)
    top2 = stack2[n2 - 1];

    // Add values
    result = bottom1 + top2;

    // Output
    printf("\nBottom element of Stack 1 = %d", bottom1);
    printf("\nTop element of Stack 2 = %d", top2);
    printf("\nResult (Sum) = %d\n", result);

    return 0;
}
```

OUTPUT:



EXP NO 34:

CODE:

```
#include <stdio.h>

// Function to swap two elements
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Heapify function
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Check left child
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // Check right child
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest not root, swap and heapify again
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Heap Sort function
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements from heap
    for (int i = n - 1; i >= 0; i--) {
        swap(&arr[0], &arr[i]); // Move current root to end
        heapify(arr, i, 0);     // Heapify reduced heap
    }
}

// Print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
```

```
    int n, i;
    int arr[50];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array using Heap Sort: ");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 6
Enter elements:
40 10 30 20 50 60
Sorted array using Heap Sort: 10 20 30 40 50 60
```

EXP NO 35:

CODE:

```
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
```

```
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Pivot element
    int i = (low - 1);

    for(int j = low; j <= high - 1; j++) {
        // If current element less than pivot
        if(arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// Quick Sort function (recursive)
void quickSort(int arr[], int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Sort left side
        quickSort(arr, pi + 1, high); // Sort right side
    }
}

// Print array
```

```
void printArray(int arr[], int n) {
   for(int i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}

int main() {
   int n, i;
   int arr[50];

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   quickSort(arr, 0, n - 1);

   printf("Sorted array using Quick Sort: ");
   printArray(arr, n);

   return 0;
}
```

OUTPUT: